Databricks vs Azure-Synapse vs Azure-Data-Factory

Criteria	Databricks Workflows	Azure Synapse Pipelines	Azure Data Factory (ADF) Pipelines
Best for	Spark-based processing, real-time analytics, machine learning pipelines	Large-scale data integration, hybrid SQL + Spark environments	ETL/ELT, data movement, orchestrating across many data sources
Data Processing Engine	Apache Spark - **Databricks Notebooks** - **Databricks Jobs** - **Delta Lake** - **MLflow**	- **SQL Pools** - **Spark Pools**	External processing (supports Spark via Databricks integration) - **Databricks Integration** - **Azure SQL Pools** - **Spark Pools**
Integration Focus	Databricks notebooks, ML, Delta Lake, Spark jobs - **Delta Lake** - **Databricks Notebooks** - **MLflow**	Synapse Analytics (SQL + Spark) - **Power BI** - **Data Explorer**	Wide-range of Azure services, databases, external APIs. - **SQL Databases** - **NoSQL Databases** - **REST APIs**
Orchestration Scope	Mostly Databricks-native jobs. - **Databricks Workflows** - **Databricks Jobs** - **Databricks Delta**	SQL, Spark, Data Explorer,PowerBI - **Data Flow** - **Notebooks** - **Power BI**	Rich data movement across diverse sources - **Data Flow** - **Control Flow**
Workload Complexity	Advanced analytics, machine learning	Combined analytics and data engineering	Complex ETL/ELT workflows with branching and looping
Connectors	Limited to Databricks-related resources	Deep integration within Synapse ecosystem	Extensive connector support (SQL, NoSQL, REST, on-premises, cloud)
Job Scheduling	Notebook and job orchestration - **Databricks Jobs** - **Databricks Workflows**	Orchestrates SQL and Spark jobs - **Synapse Pipelines** - **Data Flow**	Orchestrates multiple activities (copy, transformation, custom code) - **ADF Pipelines** - **Triggers**
Scalability	Spark-native scaling, cluster management - **Auto-scaling Clusters** - **Elastic Scaling**	- Scalability for both SQL and Spark pools - **Integrated Scaling**	Scales across multiple Azure resources (supports Databricks/Synapse) - **Scalable Pipelines** - **Dynamic Scaling**
Ease of Use	Best for Spark users familiar with Databricks - **Databricks UI** - **Notebooks for interactive analysis**	Code-free UI for SQL/Spark workloads - **Integrated SQL and Spark**	- Code-free UI - **Designed for ETL Pipelines** with extensive custom activity options.
Advanced Analytics	Excellent for machine learning and real-time analytics - **MLflow** - **Databricks SQL Analytics**	Suitable for both real-time and batch processing with SQL/Spark - **SQL Analytics** - **Spark Pools for Analytics**	More suited for batch processing and data movement - **Batch Processing** - **Data Movement**
Monitoring	Databricks-native monitoring tools - **Databricks Monitoring** - **Cluster Logs**	Synapse-integrated monitoring - **Synapse Studio Monitoring** - **Integration with Azure Monitor**	Enterprise-grade monitoring and debugging - **ADF Monitoring** - **Azure Monitor**
Pros	- Tight Spark integration	- SQL + Spark integration	- Extensive connector support
	- ML and Delta Lake support	- Unified analytics platform	- Hybrid integration
	- Auto-scaling	- Flexible ETL/ELT	- Code-free design
Cons	- Limited external orchestration	- Less mature than ADF for complex workflows - **Complex Workflow Design** - **Requires Advanced Integration**	- Less flexibility for non-Azure services - **Complex Integrations**
	- Less flexibility for non-Databricks tasks		

Layer	Databricks Workflows	Azure Synapse Pipelines	Azure Data Factory (ADF) Pipelines
Compute Model	- **Spark Clusters**	- **SQL Pools** (on-demand or provisioned)	- **Azure Integration Runtime**
	- **Auto-scaling**	- **Spark Pools**	- **Databricks Integration Runtime**
	- **Managed Compute**		
Compute Usage	- **Databricks Runtime** for Spark processing	- **SQL Pools** for large-scale SQL queries	- **Azure Integration Runtime** for data movement and transformation
	- **Managed and Scalable**	- **Spark Pools** for Spark jobs	- **Databricks Runtime** for Spark jobs
Scaling	- **Automatic Scaling** of Spark clusters	- **Dynamic Scaling** of SQL and Spark resources	- **Manual Scaling**
	- **Cluster Size Adjustment**		- **Integration Runtime scaling**
Resource Management	- **Databricks Workspaces** manage clusters	- **Dedicated SQL Pools** (for performance)	- **Self-hosted Integration Runtime** for on-premises data
	- **Cluster Pools** for efficiency	- **Shared Spark Pools**	- **Azure Integration Runtime** for cloud resources

Question:

Repartitioning or **Caching**?

In Databricks data engineering projects, when dealing with high latency and low latency data processing, the two key terms are:

1. **Repartitioning** (for high latency): This involves reorganizing the data into more or fewer partitions to optimize large-scale batch processing tasks. Repartitioning can reduce data shuffling and increase parallelism, but it is typically associated with higher latency workloads since it's used in preparation for larger, slower jobs.
2. **Caching** (for low latency): The other term is often caching, which involves storing frequently accessed or intermediate data in memory (using cache() or persist() in Spark) to reduce read times and speed up low-latency, interactive query responses.

These two techniques help balance the performance between high-latency batch processing (repartitioning) and low-latency data retrieval (caching).

Question:

What are all platforms, services or tools need to learn to perform data engineering in azure like hive , airflow etc

To perform data engineering in Azure, there are several key platforms, services, and tools you should learn. These span across data ingestion, transformation, orchestration, and storage. Here's a breakdown:

1. Data Storage & Management

- **Azure Data Lake Storage (ADLS)**: A scalable and secure storage service for storing large volumes of structured, semi-structured, and unstructured data.
- **Azure Blob Storage**: For storing unstructured data like files, videos, and logs.
- **Azure SQL Database**: A relational database service for structured data.
- **Azure Cosmos DB**: A NoSQL database service for high-availability and low-latency access to unstructured data.
- **Azure Synapse Analytics**: A unified data analytics service that integrates big data and data warehousing.

2. Data Processing & Transformation

- **Azure Databricks**: A collaborative platform for big data analytics, machine learning, and ETL (extract-transform-load) using Apache Spark.
- **Azure HDInsight**: A fully managed cloud service that makes it easy to process big data using popular open-source frameworks like Hadoop, Hive, and Spark.
- **Azure Synapse Pipelines**: Used to orchestrate data workflows and transformations similar to Azure Data Factory.

3. Data Orchestration & Workflow Management

- **Azure Data Factory (ADF)**: A cloud-based data integration service for creating ETL and ELT pipelines. It orchestrates the movement and transformation of data across various Azure services.
- **Apache Airflow** (if used with Azure): Though ADF is the native orchestration tool, Airflow can also be used for orchestration in Azure environments for advanced scheduling.

4. Data Ingestion

- **Azure Event Hubs**: For real-time ingestion of large volumes of event data from sources like IoT devices or log files.
- **Azure Stream Analytics**: For real-time analytics and data streaming.
- **Azure IoT Hub**: For IoT-based data ingestion.
- **Apache Kafka on Azure HDInsight**: For distributed streaming and event data ingestion.

5. Data Querying & Reporting

- **Azure SQL Data Warehouse / Dedicated SQL Pool**: For querying large datasets with SQL.
- **Hive on Azure HDInsight**: For querying and managing large datasets using SQL-like queries.
- **Azure Power BI**: For reporting, visualizing, and exploring data.
- **Azure Analysis Services**: For enterprise-grade data modeling.

6. Data Security & Governance

- **Azure Active Directory (Azure AD)**: For identity and access management.
- **Azure Key Vault**: For managing secrets, keys, and certificates.
- **Azure Purview**: For data governance, lineage, and cataloging.

7. Version Control & CI/CD Tools

- **Azure DevOps**: For managing project code, building CI/CD pipelines, and version control with Git.
- **GitHub Actions**: Integrated with Azure for CI/CD pipeline automation.

8. Machine Learning & AI Integration

- **Azure Machine Learning**: For building, training, and deploying machine learning models.
- **Databricks MLflow**: Used for managing machine learning lifecycle on Azure Databricks.

9. Other Tools & Frameworks

- **Apache Spark**: Core for distributed processing in Databricks and HDInsight.
- **Apache Hive**: Data warehousing tool for querying large datasets.
- **Azure Kubernetes Service (AKS)**: For containerized workload management, sometimes used for deploying scalable ETL solutions.
- **Delta Lake**: An open-source storage layer that brings ACID transactions to Apache Spark and big data workloads.
- **Azure Logic Apps**: To automate workflows between cloud services.
- **Azure Functions**: Serverless computing to run small pieces of code or tasks without managing infrastructure.

Summary:

- **Storage**: ADLS, Blob Storage, Azure SQL, Cosmos DB.
- **Data Processing**: Azure Databricks, HDInsight, Synapse Analytics.
- **Orchestration**: Azure Data Factory, Airflow.
- **Ingestion**: Event Hubs, Stream Analytics, IoT Hub.
- **Querying**: Synapse Analytics, Hive, Power BI.
- **Security**: Azure AD, Key Vault.
- **Version Control/CI**: Azure DevOps, GitHub Actions.

By mastering these tools and services, you can build robust, scalable, and efficient data engineering solutions on Azure.

Question:

Teach me hive when it comes to azure data engineering

Apache Hive is an open-source data warehouse software that allows users to read, write, and manage large datasets stored in a distributed system using SQL-like syntax, known as HiveQL (Hive Query Language). In the context of Azure Data Engineering, Hive can be used primarily on Azure HDInsight, which is a managed service for running big data frameworks like Hadoop, Hive, and Spark.

Here's a step-by-step guide to help you learn Hive for Azure data engineering:

1. Understanding Hive Architecture

In Hive, data is organized into:

- **Databases**: Collections of tables. These are logical units in Hive for better organization.
- **Tables**: Structured data stored in a database.
- **Partitions**: Logical division of a table into smaller chunks, which improves query performance.
- **Buckets**: Further sub-divisions of partitions, used to optimize performance when dealing with large datasets.

Hive abstracts the complexity of Hadoop's file storage system and allows you to use SQL queries to interact with data stored in HDFS (Hadoop Distributed File System) or Azure Data Lake Storage.

2. Hive in Azure HDInsight

Azure HDInsight provides a managed environment where you can use Hive for querying and analyzing big data. Here's how Hive fits into the Azure ecosystem:

- **Storage**: Your data can reside in **Azure Data Lake Storage (ADLS)** or **Azure Blob Storage**. HDInsight clusters can be configured to connect with these storage services.
- **Compute**: HDInsight runs on top of **Apache Hadoop**, and Hive queries are processed using **MapReduce**, **Tez**, or **Spark** as the execution engine.

3. Getting Started with Hive on HDInsight

Step 1: Provisioning an HDInsight Cluster

1. **Create an HDInsight cluster**: Go to the Azure portal, search for "HDInsight," and create a new cluster. Choose **Hive** as the primary workload.
2. **Link storage**: During cluster creation, choose **Azure Blob Storage** or **ADLS Gen2** as your default storage.

Step 2: Using Hive with the HDInsight Cluster

1. **Access Hive via SSH or Ambari**:
    - SSH into the HDInsight cluster or use **Ambari** to access Hive through a user-friendly UI.
    - You can also use **Azure Data Studio**, **Visual Studio Code**, or **Hue** to interact with Hive.
2. **Execute Hive Queries**: Once connected, you can start running Hive queries. For example:

```
-- Creating a database
CREATE DATABASE azure_data;

-- Creating a table
CREATE TABLE azure_data.employees (
  employee_id INT,
  name STRING,
  department STRING,
  salary FLOAT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

-- Loading data into the table from Azure Storage
LOAD DATA INPATH 'wasbs://<your-storage-account>@<your-container>.blob.core.windows.net/employee.csv'
```

```
INTO TABLE azure_data.employees;

-- Running a query
SELECT * FROM azure_data.employees WHERE salary > 50000;
```

4. HiveQL (Hive Query Language)

HiveQL is a subset of SQL. Here are the key operations and commands:

- **Creating Tables**: You can create both **managed** and **external** tables.
  - **Managed tables**: Hive manages the lifecycle of the table data. If you drop the table, Hive deletes the data.
  - **External tables**: You define the table schema, but Hive doesn't own the data. When you drop the table, the data remains intact.

Example of an **external table**:

```
CREATE EXTERNAL TABLE sales_data (
 transaction_id INT,
 product STRING,
 amount FLOAT,
 date STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION 'wasbs://<container>@<storage-account>.blob.core.windows.net/sales_data/';
```

- **Partitioning and Bucketing**:
  - **Partitioning**: Hive allows you to partition tables into directories based on column values. This speeds up query performance by pruning unnecessary data.

    ```
    CREATE TABLE sales_data_partitioned (
     transaction_id INT,
     product STRING,
     amount FLOAT
    )
    PARTITIONED BY (year INT, month INT)
    ROW FORMAT DELIMITED
    FIELDS TERMINATED BY ',';
    ```

  - **Bucketing**: Further divides partitions into buckets, useful for high-cardinality columns.

    ```
    CREATE TABLE sales_data_bucketed (
     transaction_id INT,
     product STRING,
     amount FLOAT
    )
    CLUSTERED BY (product) INTO 10 BUCKETS
    ROW FORMAT DELIMITED
    FIELDS TERMINATED BY ',';
    ```

- **Loading Data**: You can load data from **Azure Blob Storage** or **Azure Data Lake**.

```
LOAD DATA INPATH 'wasbs://<container>@<storage-account>.blob.core.windows.net/datafile.txt'
INTO TABLE sales_data;
```

- **Queries**: HiveQL supports most SQL operations.
    - **Basic Query**:

      ```
      SELECT product, SUM(amount)
      FROM sales_data
      GROUP BY product;
      ```

    - **Joining Tables**:

      ```
      SELECT a.transaction_id, b.product, b.amount
      FROM sales_data a
      JOIN sales_data_partitioned b
      ON a.transaction_id = b.transaction_id;
      ```

5. Optimizing Hive Queries in Azure

- **Use Partitioning**: When querying large datasets, always partition on high-cardinality columns to reduce data scan time.
- **Use ORC/Parquet Formats**: These columnar formats help improve both storage efficiency and query performance in Hive.

```
CREATE TABLE sales_data_orc (
  transaction_id INT,
  product STRING,
  amount FLOAT
)
STORED AS ORC;
```

- **Use Caching**: In case you're running Hive on **Spark execution engine**, you can cache the dataframes for better performance.

6. Hive Integration with Other Azure Services

- **Azure Data Factory (ADF)**: You can automate Hive queries by creating **pipelines** in ADF that trigger Hive scripts using the **HDInsight Hive activity**.
- **Azure Databricks**: Databricks supports **Hive metastore integration**, meaning you can query Hive tables directly from Databricks Spark clusters.
- **Power BI**: For visualization, you can connect **Power BI** to Hive tables for reporting and dashboarding.

7. Real-World Use Case Example

Imagine you have large transactional datasets stored in Azure Blob Storage. You can use Hive on HDInsight to:

1. Create external tables on top of this data.
2. Partition the tables based on transaction date.
3. Run aggregations, filtering, and joins using HiveQL.
4. Optimize the performance using ORC format and bucketing.
5. Orchestrate the process using **Azure Data Factory** to trigger Hive jobs at specific intervals, or use Airflow for more complex workflows.

Conclusion

Learning Hive in the context of Azure involves understanding how Hive fits within the Azure ecosystem, mastering HiveQL, and learning to optimize and automate data engineering workflows. By integrating Hive with other Azure services like HDInsight, ADF, and Databricks, you can create efficient, scalable, and automated data pipelines that handle big data workloads effectively.

Data Layers in Data Engineering Projects:

Layer	Purpose	Services & Tools (AWS)	Services & Tools (Azure)	Services & Tools (GCP)	Other Open-Source Tools	Use Case
1. Data Ingestion Layer	Collects raw data from various sources (batch or real-time)	- **AWS Glue**, **AWS Data Pipeline**, **AWS Kinesis**, **AWS DMS** - Streaming: **AWS Kinesis**, **Amazon MQ**	- **Azure Data Factory**, **Azure Event Hubs**, **Azure IoT Hub** - Streaming: **Azure Stream Analytics**	- **Google Dataflow**, **Google Pub/Sub**, **Google Transfer Service** - Streaming: **Google IoT Core**	- **Apache Kafka**, **Apache Flink**, **Apache NiFi**, **Logstash**	- Ingesting real-time sensor data from IoT devices using **Azure IoT Hub** or **AWS Kinesis**, transforming and loading into a data lake for further analysis.
2. Data Storage Layer	Stores raw data in different formats (structured, semi-structured, unstructured)	- **Amazon S3**, **Amazon RDS**, **Amazon DynamoDB**, **Amazon Aurora**, **Amazon Redshift**, **Amazon Neptune**	- **Azure Data Lake Storage (ADLS)**, **Azure SQL Database**, **Cosmos DB**, **Azure Synapse**	- **Google Cloud Storage**, **Google Cloud SQL**, **Google BigQuery**, **Google Spanner**, **Firestore**	- **MongoDB**, **HBase**, **Cassandra**, **PostgreSQL**, **MySQL**, **Redis**	- Storing vast amounts of structured and unstructured customer data in **Amazon S3** and performing analytics with **Redshift** or **BigQuery**.
3. Data Processing Layer	Transforms and processes data (batch or real-time)	- Batch: **AWS EMR** (with **Apache Spark**, **Hive**) - Streaming: **AWS Kinesis Data Analytics**, **AWS Glue**	- Batch: **Azure Databricks**, **Azure HDInsight**, **Azure Synapse Analytics** - Streaming: **Azure Stream Analytics**	- Batch: **Google Dataproc**, **Google Dataflow**, **Google BigQuery** - Streaming: **Google Dataflow**, **Google Pub/Sub**	- **Apache Spark**, **Apache Hive**, **Flink**, **Spark Streaming**, **Presto**, **Beam**	- Real-time data processing and analytics using **Apache Flink** or **Azure Stream Analytics** to process financial transactions or IoT sensor data in real-time.
4. Data Cleansing & Transformation Layer	Cleans, enriches, and prepares data for further analysis	- **AWS Glue**, **AWS EMR**, **AWS Lambda**	- **Azure Data Wrangling**, **Azure Databricks**, **Azure Synapse**, **Azure HDInsight**	- **Google Dataprep**, **Google Dataflow**	- **Apache Spark**, **dbt**, **Apache Hive**, **Pandas**	- Cleansing raw customer data in **Azure Databricks** or using **AWS Glue** for transformation, preparing it for advanced analytics and reporting.
5. Data Staging Layer	Holds pre-processed or cleansed data temporarily before further transformations	- **Amazon S3**, **Amazon Redshift**, **AWS Glue**	- **Azure Data Lake Storage (ADLS)**, **Azure Synapse Analytics**, **Azure Blob Storage**	- **Google Cloud Storage**, **Google BigQuery**, **Google Dataproc**	- **Apache Hive**, **Apache HBase**, **Cassandra**	- Storing intermediate data in **ADLS** before loading into **Azure Synapse** for large-scale analytics, or storing in **Amazon S3** for further transformation.
6. Data Aggregation & Analytics Layer	Aggregates data, computes metrics, and prepares it for analysis and reporting	- **Amazon Redshift**, **AWS QuickSight**, **Amazon Athena**, **AWS Glue DataBrew**	- **Azure Synapse Analytics**, **Azure Analysis Services**, **Power BI**, **Azure Databricks**	- **Google BigQuery**, **Looker**, **Google Data Studio**	- **Apache Spark**, **Apache Hive**, **Presto**, **OLAP Tools**	- Aggregating sales and customer data from multiple regions using **AWS Redshift** or **Google BigQuery**, computing insights for business decision-making.

Layer	Purpose	Services & Tools (AWS)	Services & Tools (Azure)	Services & Tools (GCP)	Other Open-Source Tools	Use Case
7. Data Orchestration Layer	Orchestrates data workflows, scheduling and monitoring data pipelines	- **Amazon Managed Workflows for Apache Airflow (MWAA)**, **AWS Step Functions**, **AWS Glue**	- **Azure Data Factory**, **Azure Logic Apps**, **Azure Functions**, **Azure Batch**	- **Google Cloud Composer** (Managed Airflow), **Google Cloud Functions**, **Google Workflows**	- **Apache Airflow**, **Prefect**, **Dagster**, **Apache NiFi**	- Using **Apache Airflow** or **Azure Data Factory** to orchestrate end-to-end ETL workflows, including data ingestion, transformation, and loading into the data warehouse.
8. Data Serving Layer	Makes processed data available to end users and applications	- **Amazon Redshift**, **Amazon RDS**, **AWS API Gateway**, **Amazon Athena**, **AWS Lambda**	- **Azure Synapse Analytics**, **Azure SQL Database**, **Cosmos DB**, **Azure API Management**	- **Google BigQuery**, **Google Cloud SQL**, **Google Firestore**, **Google API Gateway**	- **PostgreSQL**, **MySQL**, **Cassandra**, **MongoDB**	- Serving processed analytics results to business applications or BI tools using **Azure Synapse** or **Google BigQuery**, making data available for querying via APIs.
9. Data Presentation & Visualization Layer	Visualizes and reports data for decision-making and business intelligence	- **AWS QuickSight**, **Amazon Athena**	- **Power BI**, **Azure Analysis Services**, **Azure Synapse Analytics**	- **Looker**, **Google Data Studio**, **Google BigQuery**	- **Tableau**, **Apache Superset**, **Qlik**	- Visualizing sales performance and customer behavior with **Power BI**, **Tableau**, or **Google Data Studio**, creating dashboards for key business metrics.
10. Data Governance, Security & Monitoring Layer	Ensures data quality, security, governance, and compliance, and monitors the pipeline	- **AWS Glue Data Catalog**, **AWS Lake Formation**, **AWS IAM**, **Amazon CloudWatch**, **Amazon Macie**	- **Azure Purview**, **Unity Catalog (Databricks)**, **Azure Monitor**, **Azure AD**, **Azure Key Vault**, **Azure Sentinel**	- **Google Cloud Data Catalog**, **Google Cloud IAM**, **Google Cloud Monitoring**, **Google Security Command Center**, **Google Key Management**	- **Apache Ranger**, **Apache Atlas**, **Collibra**, **Apache Knox**	- Enforcing data governance policies across a multi-cloud environment using **Unity Catalog** in Databricks or **AWS Glue Data Catalog** for security and governance.
11. Data Archival Layer	Archives data for long-term storage and compliance	- **Amazon S3 Glacier**, **AWS Backup**	- **Azure Blob Storage (Cool/ Archive Tier)**, **Azure Backup**	- **Google Cloud Archive**, **Google Cloud Storage**	- **Apache HDFS**, **Tapestry**	- Archiving historical data that is no longer in active use but required for compliance (e.g., customer records) in **AWS Glacier** or **Azure Blob Archive**.

Key Use Cases Explained by Layer:

1. **Data Ingestion Layer**:
   - **Use Case**: A company collects real-time IoT sensor data using **Azure IoT Hub** or **AWS Kinesis** and loads it into a data lake (ADLS or S3) for further processing.

2. **Data Storage Layer**:
   - **Use Case**: An e-commerce platform stores customer transaction data in **Amazon S3** for raw data storage and uses **Amazon DynamoDB** to store structured, queryable data on user sessions.

3. **Data Processing Layer**:

- **Use Case**: A retail business processes real-time clickstream data using **Apache Flink** on **AWS EMR** to identify customer behavior trends in near real-time.

4. **Data Cleansing & Transformation Layer**:
   - **Use Case**: A financial institution uses **AWS Glue** to cleanse and transform transaction data, preparing it for fraud detection analysis.

5. **Data Staging Layer**:
   - **Use Case**: A media company stores cleansed customer data temporarily in **Azure Data Lake Storage** before loading it into **Azure Synapse** for advanced analytics.

6. **Data Aggregation & Analytics Layer**:
   - **Use Case**: A global retailer aggregates daily sales data across regions using **AWS Redshift** and computes key metrics such as revenue growth for executive reporting.

7. **Data Orchestration Layer**:
   - **Use Case**: An insurance company orchestrates its data pipeline using **Apache Airflow** to automatically pull policyholder data from multiple sources, transform it, and load it into a centralized database.

8. **Data Serving Layer**:
   - **Use Case**: An online service makes processed user data available through **Google BigQuery** for analysts and through an API for customer-facing apps using **Google API Gateway**.

9. **Data Presentation & Visualization Layer**:
   - **Use Case**: A marketing team uses **Power BI** to visualize campaign performance, presenting insights on user engagement and conversion rates in real-time dashboards.

10. **Data Governance, Security & Monitoring Layer**:
    - **Use Case**: A healthcare organization ensures patient data compliance using **Azure Purview** or **Unity Catalog** for governance and access control, ensuring sensitive data is handled according to regulations.
11. **Data Archival Layer**:
    - **Use Case**: A law firm archives case files and legal records that are no longer actively used but must be retained for legal compliance in **AWS Glacier**.

This table provides a comprehensive overview of how different tools and services can be applied across cloud platforms and layers in real-world data engineering projects.